

Двухуровневый алгоритм: сначала формально разобрать строку, потом применить правило исключения символов. Ответ для исходной фразы остаётся «И» [1][2][3].

Алгоритм

Идея такая:

Вход — строка с фразой

Нормализуем текст

Находим имена/буквы, которые “сели на трубе”

Применяем действия из следующей части фразы: “А упала, Б пропала”

Убираем их из множества

Выводим то, что осталось

Формально:

$$S_0 = \{A, И, Б\}$$

$$S_1 = S_0 \setminus \{A\}$$

$$S_2 = S_1 \setminus \{Б\}$$

$$\text{Answer} = S_2 = \{И\}$$

Python-реализация

python



```
import re
```

```
def solve_riddle(text: str):
```

```
    text = text.upper()
```

```
    seated = set()
```

```
    m = re.search(r"([А-ЯА-ZЁИ])+\s+И\s+
```

```
([А-ЯА-ZЁİİ]+)\s+СИДЕЛИ?
```

```
\s+НА\s+ТРУБЕ", text)
```

```
if m:
```

```
    seated.add(m.group(1))
```

```
    seated.add("И")
```

```
    seated.add(m.group(2))
```

```
fallen = re.findall(r"([А-ЯА-ZЁİİ]+)\s+  
(?:УПАЛ[АО]?|ПРОПАЛ[АО]?)", text)
```

```
for x in fallen:
```

```
    seated.discard(x)
```

```
if len(seated) == 1:
```

```
    return next(iter(seated))
```

```
return sorted(seated)
```

```
# Пример
```

```
riddle = "А и Б сидели на трубе. А  
упала, Б пропала. Кто остался на  
трубе?"
```

```
print(solve_riddle(riddle))
```

Что делает этот код

Он не просто “знает ответ”, а распознаёт структуру загадки как маленькую языковую модель:

выделяет множество персонажей;

выделяет события удаления;

возвращает оставшийся элемент [4][5].

Уникальная версия

ОБЩАЯ функция, которая будет понимать любые загадки вида:

“X, Y и Z сидели на трубе”

“X упала”

“Y пропала”

Тогда алгоритм станет не частным решением, а универсальным интерпретатором считалок-исключений.

Более обобщённый вариант

python



```
import re
```

```
def parse_pipe_riddle(text: str):  
    text = text.upper()
```

```
    people = re.findall(r"\b[A-ЯA-ZЁİİ]+\b",  
text)
```

```
    seated = []
```

```
    first_part = re.search(r"(.+?)  
\s+СИДЕЛ[ИА]? \s+НА \s+ТРУБЕ", text)  
    if first_part:
```

```
prefix = first_part.group(1)
seated = re.findall(r"\b[A-ЯА-ZЁӀӐ]+
\b", prefix)
```

```
fallen = re.findall(r"\b([A-ЯА-ZЁӀӐ]+)
\b\s+(?:УПАЛ[АО]?|ПРОПАЛ[АО]?)",
text)
```

```
result = [p for p in seated if p not in
fallen]
return result[0] if len(result) == 1 else
result
```

```
print(parse_pipe_riddle("А и Б сидели
на трубе. А упала, Б пропала. Кто
остался на трубе?"))
```

